STERLING · SBG FUNDING AI GUIDE

Interview prep

docs/INTERVIEW_PREP.md · October 29, 2026

The 30-second version

> SBG's website has product pages, two calculators, an FAQ and a phone number, but nothing that walks a business owner through them. I built Sterling, a prototype of that guide. It asks the questions an advisor would ask, checks the answers against SBG's published minimums, runs SBG's own calculator model, answers questions from SBG's content with citations, and hands off to an advisor with the whole conversation attached. The design principle is "code decides, the model explains": eligibility and dollar figures come from tested code, and Claude handles language: understanding free text, picking tools and explaining results.

The demo (about 3 minutes)

Before the interview:

- Start the production build (`npm run build && npm start`) with your API key in `.env.local`. Open http://localhost:3000 and `/specialist` in two tabs.
- Run one conversation in a private window that ends with *"I can't make this week's payment on my advance"*, so the inbox already holds a high-priority hardship ticket to show.
- Then click "Start over" in the main tab so you begin from the welcome screen.

Time	Do	Say
0:00	Welcome screen	"This is a prototype of a guide for SBG's site. It uses SBG's colors and fonts, so it could sit on their homepage."
0:15	Type: *I run a roofing company, an LLC, 3 years in business, about $80K a month*	"One sentence, four fields. Claude calls a tool that fills a structured profile, and the panel highlights exactly what it captured."
0:40	Answer the cards: Equipment, $100K, Within a week, 680–719	"The question order comes from a state machine, not the model, so the flow is predictable and works even without the model."
1:05	Results card	"These checks are SBG's four published minimums, applied by deterministic code. The same profile always gets the same answer, which matters in lending."
1:25	Estimate card: drag the slider	"This is a port of the script behind SBG's own calculator, so the numbers match their website. It's unit-tested against hand-computed values."

Time	Do	Say
1:45	Type: *How fast can equipment financing fund?*	"Facts come from retrieval over SBG's content: hybrid keyword and semantic search, cited inline. If it can't find something, it says so and offers a person."
2:05	Open "Under the hood"	"Every turn is traced: guardrails, retrieval scores, tool calls, model, tokens, latency. It's how I debugged it, and how you'd monitor it."
2:25	Click "Talk to a specialist", then switch to the inbox tab	"The advisor gets the profile, a summary to read first and the transcript, so the customer never repeats themselves. Hardship and complaints are matched by rules before the model sees them, and they're marked high priority."
2:50	Close	"Without an API key it still works in retrieval-only mode. There are 100 tests, and retrieval and routing evals are in the README."

If anything breaks: the guided flow, results, estimate and handoff don't need the model. If the model fails, the inspector shows the fallback and the app keeps working. Screenshots of every screen are in `docs/screenshots/` as a backup.

How to talk about how you built it

Be straightforward about using AI coding tools. For a role whose projects include "chatbot development and AI applications", directing AI tools well is part of the job, and interviewers increasingly expect it. What they're really testing is whether you understand and can defend the system. Don't claim you typed every line; if they ask, a clear answer is stronger:

> "I used an AI coding assistant heavily, the way I'd expect to on the job. I owned the problem and the decisions: I picked the use case from SBG's site, set the requirements (grounded answers, guided intake matching SBG's process, human handoff, voice, SBG's look), pushed for the research into SBG's and third-party sources, and reviewed and tested the result. I can walk you through any part of it."

Then prove it by being able to explain any file. That's what the study guide is for.

System design questions

Walk me through the architecture. Browser → `/api/chat` (stateless, streams NDJSON) → orchestrator. Input guardrails run first: PII redaction, then rule routing for sensitive messages. Free text goes to a Claude tool loop with five tools: retrieval, profile update, eligibility, calculator and escalation. Button presses skip the model and go straight to the guided-flow state machine. Handoffs write tickets that advisors see in an inbox. Draw the diagram from the README.

Why RAG instead of fine-tuning? SBG's terms change, and answers need citations. Fine-tuning teaches style and patterns, not reliable, updatable facts; changing a fact would mean retraining, and you can't cite a weight. RAG updates by re-ingesting a document, shows its sources, and lets the model abstain when nothing matches.

How do you prevent hallucinations? In layers. Facts come only from retrieved passages, which are cited. Decisions and numbers come from deterministic tools, never the model. A confidence gate abstains when retrieval is weak. An output check catches approval language. Evals measure retrieval quality. And the prompt treats "I don't know, here's a specialist" as a good answer.

Why not let the model decide who qualifies? Correctness, consistency, testability and auditability. Fair-lending rules (ECOA / Regulation B) mean treating applicants consistently and being able to explain decisions. A deterministic engine gives the same answer for the same inputs, is unit-tested, and changes in one line when SBG changes a minimum.

Why hybrid search? Why RRF? Embeddings handle paraphrase; BM25 handles exact tokens like phone numbers, "7(a)" and legal names. Their scores aren't comparable (cosine 0–1, BM25 unbounded), so RRF fuses by rank. The BM25 weight of 0.3 came from a sweep on the eval set. Hybrid improved hit@3 from 89.1% to 93.5%.

How would this scale to 100,000 conversations a day? The server is stateless, so it scales horizontally behind a load balancer with no session store. The bottlenecks are model rate limits and cost, not compute. Most intake steps are button presses that never call the model, the static prompt prefix is cached, and effort is set low for chat. Background work like AI summaries moves to a queue. Retrieval is in-process and fast at this size; at thousands of documents, move to pgvector with an HNSW index.

What's the latency budget? Guided steps take a few milliseconds (no model). For typed questions, the first token arrives about as fast as the model starts streaming. Tool status lines show immediately ("Searching SBG's knowledge base…") so the wait feels purposeful. Local embeddings take tens of milliseconds. Effort is low to keep thinking short.

How would you evaluate it in production? Offline: golden sets built from real redacted conversations, run in CI on every prompt, model or content change, with regressions blocking merges. Measure retrieval (hit@k, MRR), routing (precision and recall per route) and groundedness, using an LLM grader calibrated on human labels. Online: intake completion, apply-click rate, escalation rate by reason, fallback rate, retrieval confidence distribution, cost per conversation, and advisor ratings of handoff quality. Plus weekly human review of sampled conversations.

What about security and privacy? PII (SSNs, cards via Luhn, account numbers) is redacted before the model or storage sees it. Sensitive routes bypass the model. Tool inputs are schema-validated. Client state is revalidated every turn. Prompt injection is flagged, and more importantly it can't do damage: the model can't approve anything or change numbers, because those come from code. In production: SSO on the inbox, rate limiting and bot protection, secrets in a vault, retention limits.

What about compliance? Sterling shows estimates, never offers, and says so. Commercial financing disclosure laws (New York's and California's, for example) apply to actual offers, which advisors make. Fair lending is covered above. Callback requests need TCPA consent language. All customer-facing copy would go through compliance review.

How do you keep the knowledge base current? A CMS webhook triggers re-ingest when content changes. The index is versioned, so any answer can be traced to a content version. The eval suite gates each new index before it goes live.

What happens when Claude is down? The SDK retries twice. After that, that turn runs in retrieval-only mode: the same flow, eligibility, estimates and handoff, with extractive answers. Safety refusals get a server-side fallback model, then a polite redirect. The user always has a path forward.

How do you handle long conversations? Only visible text is sent back across turns, capped at 24 turns. The structured profile carries the important facts forward in `<app_state>`, so nothing important lives only in old messages. For much longer sessions, summarize older turns.

How would you add a product or change a rule? Edit `lib/sbg/products.ts` and the rules in `lib/engine/eligibility.ts`, add a test, add a section to the knowledge base, then run `npm run kb:pdf && npm run ingest && npm run eval`.

What would you build next?

1. Evals from real conversations.
2. CRM integration for tickets.
3. Query rewriting to fix the known misses ("Do you lend in Texas?").
4. A reranker.
5. Sign-in for existing customers so it can answer account questions.
6. Analytics on where people drop off.

What was the hardest bug? The hardship routing bug (STAR story 2 below) is the best story. A runner-up: a scroll bug where smooth scrolling fired scroll events that the code read as the user scrolling away, so replies stopped partway on phones. The fix: only an upward scroll counts as the reader leaving.

Behavioral: stories from this project

Use STAR: situation, task, action, result. Keep each to about 90 seconds. Also prepare two or three stories from your own past work, school or jobs, for questions about teamwork and conflict; this project mostly covers ownership, judgment and problem-solving.

1. Conflicting information (ambiguity, judgment)

- *Situation:* Researching SBG, third-party reviews disagreed with SBG and with each other. Bankrate reported a $400,000 minimum annual revenue; Finder $250,000; others $180K–$350K. SBG publishes $150,000. ConsumerAffairs said there's a hard credit pull before funding; SBG says soft pull only.

- *Task:* Decide what the chatbot should tell customers.
- *Action:* Made SBG's own current published terms the source of truth, documented the conflicts, and had the knowledge base say third-party figures vary and should be confirmed with an advisor.
- *Result:* Every answer traces to a source SBG controls, and the conflicts are documented for whoever maintains it.

2. Finding and fixing a bug systematically

- *Situation:* While seeding demo data, "I can't make this week's payment on my current advance" was routed to account servicing instead of payment hardship, losing its high priority.
- *Action:* Traced it to two causes in the routing rules. The pattern allowed only one word before "payment", so "this week's payment" didn't match. And phones type curly apostrophes (’), which none of the rules matched. I added the failing messages to the evaluation set first, then fixed the pattern and normalized apostrophes before matching.
- *Result:* All 35 routing cases pass, including the new ones. The lesson: evaluation sets should grow from real failures.

3. Product judgment (customer focus, responsible AI)

- *Situation:* Handoff tickets listed "best-fit products" for the advisor. A hardship ticket listed a term loan, a line of credit and SBA.
- *Action:* Recognized that an advisor helping someone who can't make a payment shouldn't be prompted to sell them more debt. Changed servicing tickets (hardship, complaints, existing customers) to carry no product suggestions.
- *Result:* The handoff matches the situation: support for struggling customers, sales context only for sales leads.

4. A design trade-off (technical judgment)

- *Situation:* The simplest build would have let the model decide eligibility and compute estimates from instructions in the prompt.
- *Action:* Put decisions in deterministic, tested code and exposed it to the model as tools, porting SBG's calculator script exactly, including a quirk at the $8,000 revenue boundary.
- *Result:* Consistent, explainable, testable answers that match SBG's own website. The model does what it's good at: language.

5. Testing caught logic errors

- *Situation:* Writing scenario tests for the eligibility engine.
- *Action:* Before writing assertions, I printed the engine's ranking for each scenario and checked it against what an advisor would expect. Two things were off. The rule "show the HELOC when nothing else fits" counted the HELOC itself among "everything", so it only fired when the HELOC was blocked too. And SBA 7(a) could never rank first, because a caveat every customer gets (more paperwork) was treated as a sign of poor fit.

- *Result:* Fixed both and locked them in with tests. SBA now leads for a flexible-timeline acquisition, its best use case, and drops below faster products when the customer needs money now.

Questions to ask them

- Where do applicants drop off today, and how many visitors call rather than apply online?
- How do advisors qualify a lead now, and what do they wish they knew before the first call?
- How does compliance review customer-facing AI features?
- What does the data and evaluation setup look like for the chatbot and model projects?
- What would a great first month look like for an intern on the chatbot project?

Numbers to know

Sterling	
Knowledge base	9 source docs, 73 sections, 33-page PDF, 105 chunks
Chunks	max 150 words, structure-aware, with section context
Embeddings	all-MiniLM-L6-v2, 384 dimensions, local
Fusion	RRF, K = 60, weights dense 1 / BM25 0.3
Confidence	high ≥ 0.50 cosine, low < 0.33
Retrieval eval	hybrid hit@3 93.5%, hit@5 95.7%, MRR 0.839 (46 questions)
Routing eval	35 cases, all correct
Tests	100
Agent	5 tools, up to 6 steps, history capped at 24 turns

SBG Funding	
Minimums	500+ FICO, 6+ months, $150K+ yearly revenue, 4 bank statements
Range	$5K to $10M, all 50 states
Scale	$3B+ funded since 2017, 28,000+ businesses
Contact	844-284-2725, Mon–Fri 9am–6pm ET
Model	Direct lender for revenue-based financing; marketplace for the other products